

Universidad Nacional de Ingeniería



Área del Conocimiento de Tecnología de la Información y Comunicación

Ingeniería de Sistemas

Diseño de Sistemas en Internet

Nombre:

Jafet Alexander Aguilar Martínez

Carnet:

2022-0848U

Tema:

GD_S7_S1, S2, S1F

Grupo:

5T1-SIS

Docente:

Ing. Cristopher Larios

Guía 1

1.Introducción y objetivo

Este documento registra el desarrollo paso a paso de la Guía Didáctica N° 1 de la asignatura Diseño de sistemas en Internet. El objetivo es comprender los conceptos esenciales de NestJS a través de la creación de un primer proyecto, su ejecución en un servidor local, la conexión con una base de datos mediante Prisma ORM y la definición de un esquema con relaciones entre tablas.

Cada apartado incluye la explicación del concepto, los comandos utilizados y las capturas de pantalla que evidencian el resultado obtenido en el equipo de trabajo.

Enlace de Repositorio de Github: https://github.com/Jafet2004/Guias-DSI_5T1-SIS-S.git

2. Ecosistema utilizado

2.1 Node.js

Node.js es un entorno de ejecución de JavaScript multiplataforma y de código abierto. Ejecuta el motor V8 (el núcleo de Google Chrome) fuera del navegador, lo que le da un alto rendimiento. Una aplicación Node.js se ejecuta en un solo proceso, sin crear un nuevo hilo por cada solicitud, y ofrece primitivas de E/S asíncronas que evitan bloquear el código JavaScript; por ello, el comportamiento bloqueante es la excepción y no la norma.

2.2 NestJS

NestJS es un framework para construir aplicaciones de servidor con Node.js, escrito con TypeScript y organizado en módulos, controladores y proveedores (providers) que se conectan mediante inyección de dependencias. Su documentación oficial se encuentra en <https://docs.nestjs.com/>.

3. Verificación de versiones e instalación

Antes de crear el proyecto se comprobó que Node.js y Nest CLI estuvieran instalados. Si Node.js no está presente debe instalarse desde <https://nodejs.org/es/download>, y el CLI de Nest se instala de forma global.

```
node -v
npm i -g @nestjs/cli
nest -v
```

En el equipo se verificaron las versiones **Node.js v22.16.0** y **Nest CLI 12.0.8** (Figura 1).

4. Creación del proyecto

Desde la carpeta de trabajo (GuiasDSI) se abrió la terminal y se ejecutó el comando de creación con las iniciales del autor:

```
nest new my_proyect_JAAM
```

```

● PS C:\Users\MASTER\Desktop> cd GuiasDSI
● PS C:\Users\MASTER\Desktop\GuiasDSI> node -v
v22.16.0
● PS C:\Users\MASTER\Desktop\GuiasDSI> nest -v
12.0.8
○ PS C:\Users\MASTER\Desktop\GuiasDSI> nest new my_proyect_JAAM
  ✨ We will scaffold your app in a few seconds..

? Which package manager would you ❤️ to use?
> npm
  yarn
  pnpm
  bun

↑↓ navigate • ↵ select

```

Figura 1. Verificación de versiones y ejecución de nest new; selección del administrador de paquetes.

Durante el asistente se seleccionaron las siguientes opciones:

- **Administrador de paquetes:** npm (recomendado en la guía).
- **@nestjs/observe** (trazas distribuidas, logs correlacionados y métricas): se aceptó la opción "Yes".
- **Sistema de módulos:** CJS (CommonJS) con Jest para pruebas.

```

○ PS C:\Users\MASTER\Desktop\GuiaDSI-571-SIS-S> nest new my_proyect_JAAM
  ✨ We will scaffold your app in a few seconds..

✓ Which package manager would you ❤️ to use? npm
✓ Would you like to set up @nestjs/observe (distributed tracing, auto-correlated logs, metrics and alarms - free for 300k events/month, https://observe.nestjs.com)? Yes
✓ Which module system would you like to use? CJS (CommonJS) [ with jest ]

CREATE my_proyect_jaam/.prettierrc (56 bytes)
CREATE my_proyect_jaam/jest.config.ts (878 bytes)
CREATE my_proyect_jaam/nest-cli.json (179 bytes)
CREATE my_proyect_jaam/oxlint.json (269 bytes)
CREATE my_proyect_jaam/package.json (1559 bytes)
CREATE my_proyect_jaam/README.md (6665 bytes)
CREATE my_proyect_jaam/tsconfig.build.json (178 bytes)
CREATE my_proyect_jaam/tsconfig.json (593 bytes)
CREATE my_proyect_jaam/src/app.controller.ts (286 bytes)
CREATE my_proyect_jaam/src/app.module.ts (712 bytes)
CREATE my_proyect_jaam/src/app.service.ts (150 bytes)
CREATE my_proyect_jaam/src/main.ts (299 bytes)
CREATE my_proyect_jaam/src/app.controller.spec.ts (639 bytes)
CREATE my_proyect_jaam/test/jest-e2e.json (192 bytes)
CREATE my_proyect_jaam/test/app.e2e-spec.ts (754 bytes)

➡ Installation in progress... 🌀

```

Figura 2. Opciones elegidas en el asistente y archivos generados por Nest CLI.

Nest CLI generó los archivos base del proyecto (por ejemplo `package.json`, `nest-cli.json`, `tsconfig.json`, `src/main.ts`, `src/app.module.ts`, `src/app.controller.ts` y `src/app.service.ts`) e instaló las dependencias.

```
✓ Installation in progress... 🍷

🔧 Successfully created project my_project_jaam
👉 Get started with the following commands:

$ cd my_project_jaam
$ npm run start

🌟 @nestjs/observe is wired up. Create a free account to get your credentials:

https://observe.nestjs.com
Then paste the app key and secret into ObserveModule.forRoot() in your AppModule.

Thanks for installing Nest 🙌
Please consider donating to our open collective
to help us maintain this package.

👉 Donate: https://opencollective.com/nest

PS C:\Users\MASTER\Desktop\GuiaDSI-5T1-SIS-S>
```

Figura 3. Proyecto creado correctamente e indicaciones para iniciarlo.

5. Ejecución de la aplicación

Una vez creado el proyecto se ingresa a su carpeta y se inicia en modo desarrollo con recarga automática (watch):

```
cd my_project_JAAM
npm run start:dev
```

```
PS C:\Users\MASTER\Desktop\GuiaDSI-5T1-SIS-S> cd my_project_JAAM
PS C:\Users\MASTER\Desktop\GuiaDSI-5T1-SIS-S\my_project_JAAM> npm run start:dev

> my_project_jaam@0.0.1 start:dev
> nest start --watch
```

Figura 4. Ingreso a la carpeta del proyecto y ejecución de npm run start:dev.

La consola muestra la compilación sin errores y el arranque de los módulos de la aplicación. Se observa el mapeo de la ruta / (GET) del `AppController` y el mensaje final “Nest application successfully started”.

```
[12:22:40] Starting compilation in watch mode...

[12:22:42] Found 0 errors. Watching for file changes.

[Nest] 4476 - 28/09/2026, 12:22:43 LOG [NestFactory] Starting Nest application...
[Nest] 4476 - 28/09/2026, 12:22:43 LOG [InstanceLoader] DiscoveryModule dependencies initialized +14ms
[Nest] 4476 - 28/09/2026, 12:22:43 LOG [InstanceLoader] AppModule dependencies initialized +1ms
[Nest] 4476 - 28/09/2026, 12:22:43 LOG [InstanceLoader] ObserveModule dependencies initialized +0ms
[Nest] 4476 - 28/09/2026, 12:22:43 LOG [RoutesResolver] AppController {/}: +9ms
[Nest] 4476 - 28/09/2026, 12:22:43 LOG [RouterExplorer] Mapped {/, GET} route +3ms
[Nest] 4476 - 28/09/2026, 12:22:43 LOG [ObserveAgentWorker] Observe agent worker initialized.
[Nest] 4476 - 28/09/2026, 12:22:43 LOG [NestApplication] Nest application successfully started +3ms
```

Figura 5. Salida de consola: compilación sin errores y aplicación iniciada.

6. Validación con cliente de APIs

Para comprobar que el servidor responde se realizó una petición GET a `localhost:3000`. La guía menciona Postman; en este caso se utilizó **Bruno** (colección Consultas_Bruno). La respuesta fue **200 OK** con el texto “Hello World!”, lo que confirma que la aplicación se ejecuta en el servidor local, puerto 3000.

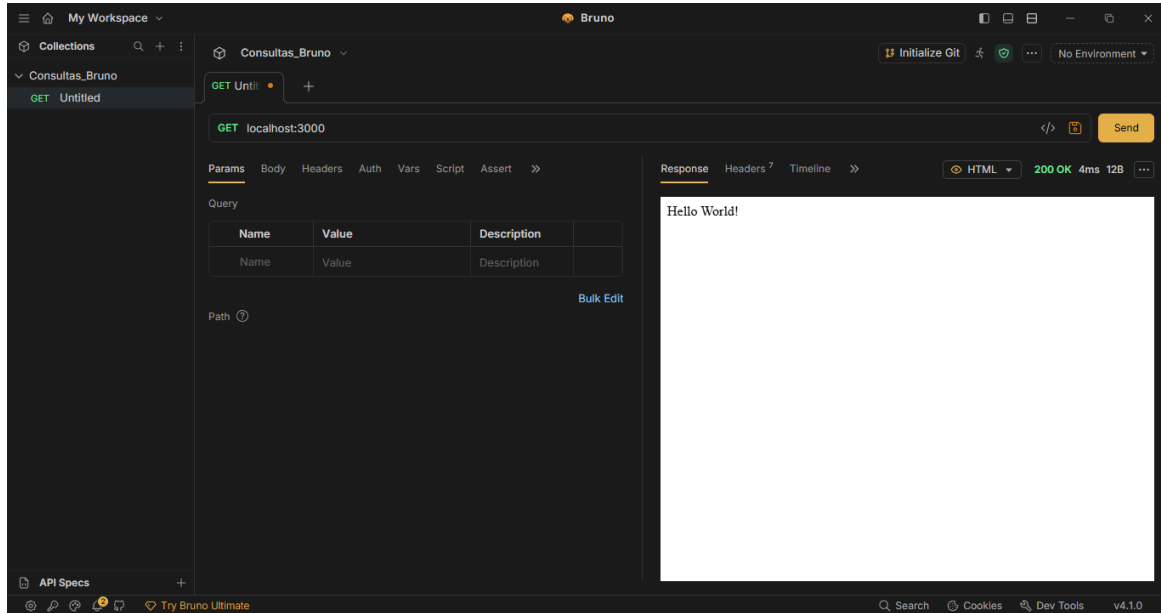


Figura 6. Petición GET a localhost:3000 en Bruno con respuesta 200 OK – “Hello World!”.

7. Estructura del proyecto

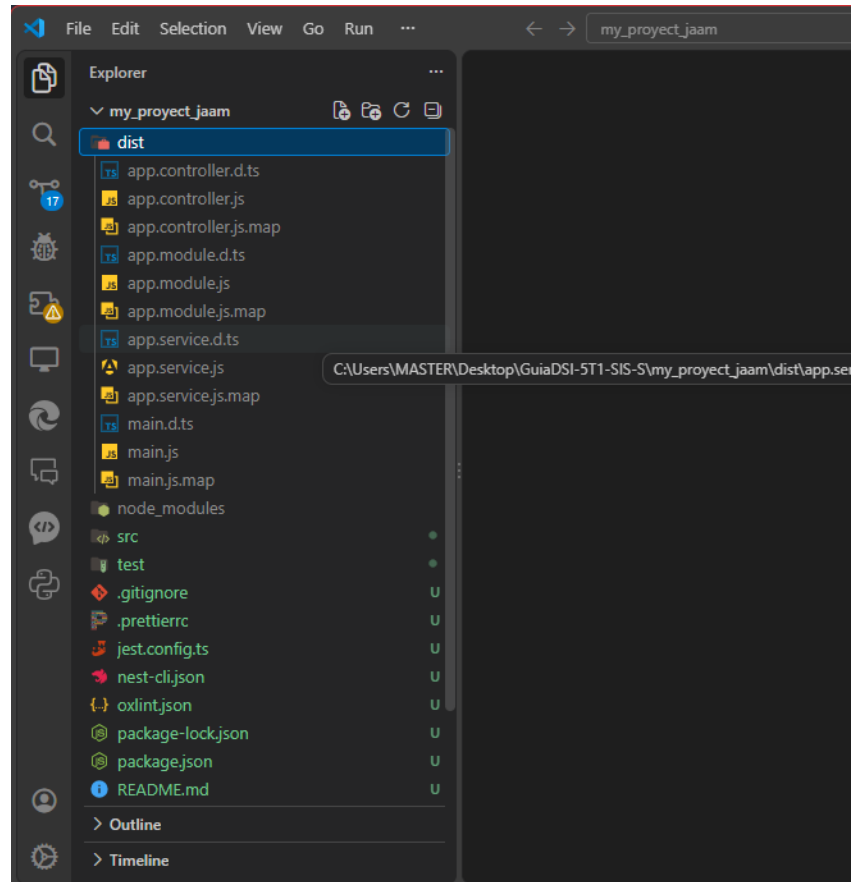


Figura 7. Estructura de carpetas y archivos del proyecto en Visual Studio Code.

- **dist:** contiene el código del proyecto una vez compilado a JavaScript.
- **node_modules:** almacena las dependencias que el proyecto necesita para funcionar.
- **src:** carpeta de código fuente y carpeta principal; a partir de ella se crean los recursos de la aplicación.
- **test:** espacio para configurar pruebas del código.
- **.gitignore:** indica las carpetas y archivos que Git debe ignorar en el seguimiento de cambios.
- **package.json y package-lock.json:** registran las dependencias necesarias e instaladas; a partir de ellos se genera `node_modules`.
- **nest-cli.json, tsconfig.json y jest.config.ts:** configuración de Nest CLI, TypeScript y Jest respectivamente.

8. Elementos clave de NestJS

La guía recomienda revisar los videos sobre inyección de dependencias y decoradores antes de continuar. El proyecto base ya incluye los tres elementos fundamentales de Nest:

- **Controladores (`app.controller.ts`):** reciben las solicitudes HTTP y devuelven la respuesta al cliente. En el proyecto base, el controlador atiende la ruta GET `/`.

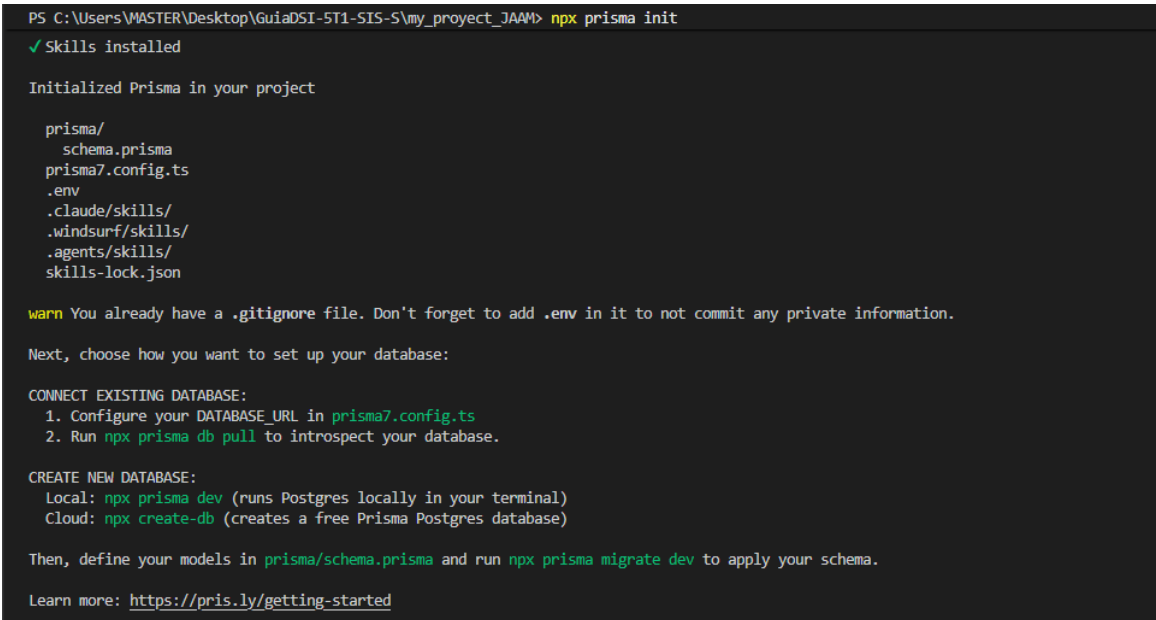
- **Providers / servicios (`app.service.ts`)**: contienen la lógica de negocio y se inyectan en los controladores mediante inyección de dependencias.
- **Módulos (`app.module.ts`)**: agrupan controladores y providers y permiten organizar la aplicación. El módulo raíz es AppModule.

9. Esquema y migraciones con Prisma

Prisma es el ORM utilizado para definir el esquema de la base de datos y generar las migraciones. La guía propone PostgreSQL como ejemplo; en este proyecto se utilizó **MySQL**, ya que la elección del origen de datos depende de las necesidades del diseñador del software.

9.1 Inicialización de Prisma

```
npm i prisma -D
npx prisma init
```



```
PS C:\Users\MASTER\Desktop\GuiaDSI-ST1-SIS-S\my_proyect_JAAM> npx prisma init
✓ Skills installed

Initialized Prisma in your project

prisma/
  schema.prisma
prisma7.config.ts
.env
.claude/skills/
.windsurf/skills/
.agents/skills/
skills-lock.json

warn You already have a .gitignore file. Don't forget to add .env in it to not commit any private information.

Next, choose how you want to set up your database:

CONNECT EXISTING DATABASE:
1. Configure your DATABASE_URL in prisma7.config.ts
2. Run npx prisma db pull to introspect your database.

CREATE NEW DATABASE:
Local: npx prisma dev (runs Postgres locally in your terminal)
Cloud: npx create-db (creates a free Prisma Postgres database)

Then, define your models in prisma/schema.prisma and run npx prisma migrate dev to apply your schema.

Learn more: https://pris.ly/getting-started
```

Figura 8. Resultado de npx prisma init: se crean la carpeta prisma, el archivo .env y prisma7.config.ts.

El comando crea la carpeta `prisma/` con el archivo `schema.prisma`, el archivo `.env` con la variable de conexión y el archivo de configuración `prisma7.config.ts`. Además advierte que `.env` debe agregarse al `.gitignore` para no publicar información privada.

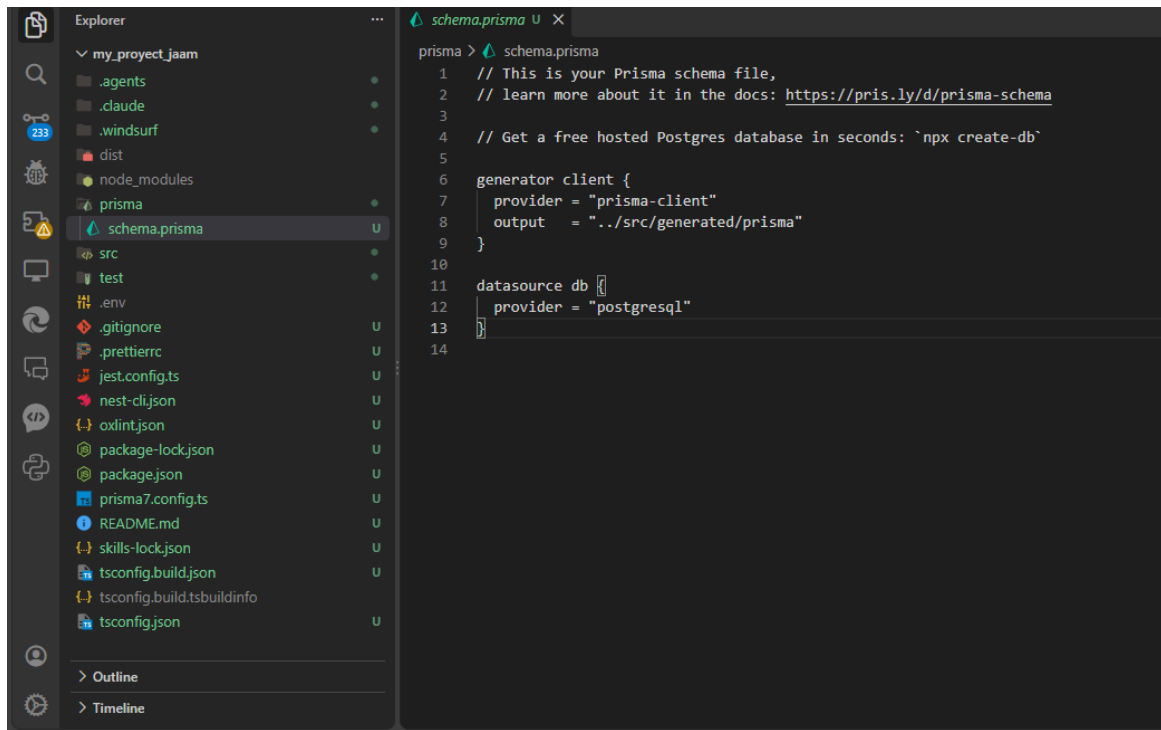


Figura 9. Archivo schema.prisma generado con los bloques generator y datasource.

El archivo schema.prisma está escrito en el lenguaje de esquema de Prisma y contiene tres elementos principales:

- **Generator:** crea el cliente que facilita manipular la base de datos desde el código (prisma-client, con salida en ../src/generated/prisma).
- **Datasource:** conecta la aplicación con el origen de datos configurado.
- **Model:** define las tablas (modelos) de la base de datos.

Se recomienda instalar la extensión de Prisma en el editor de código y usar la herramienta <https://prisma-erd.simonknott.de/> para visualizar el diagrama del esquema conforme se avanza.

9.2 Configuración del origen de datos (MySQL)

En schema.prisma se cambió el proveedor a mysql y se referenció la variable de entorno de conexión:

```

datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

```

```

12 datasource db {
13   provider = "mysql"
14   url      = env("DATABASE_URL")
15 }

```

Figura 10. Datasource configurado con el proveedor mysql.

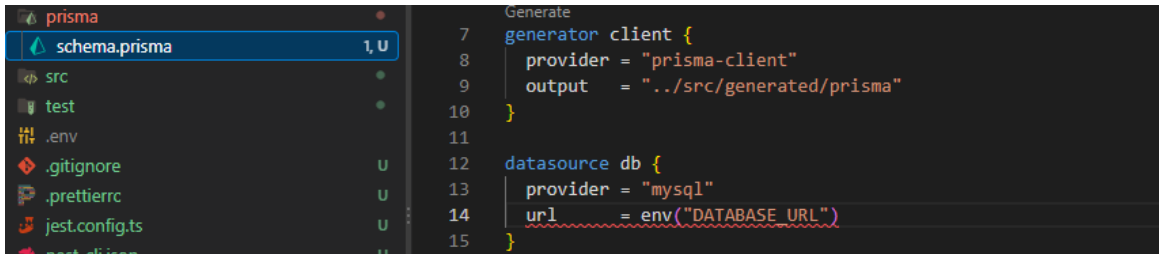


Figura 11. Bloques generator y datasource en schema.prisma.

En el archivo .env se definió la cadena de conexión con el formato

mysql://usuario:contraseña@localhost:3306/nombre_bd:

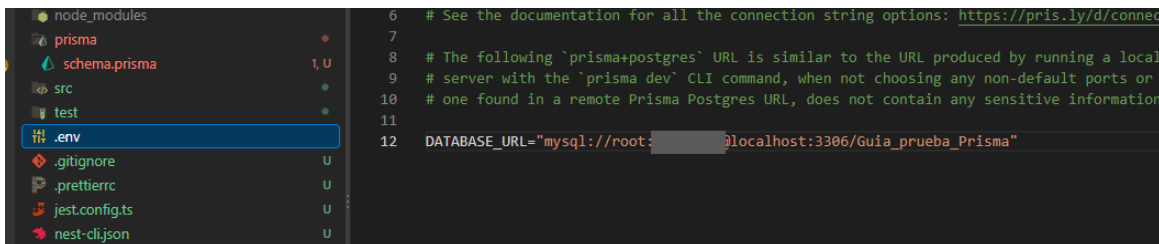


Figura 12. Variable DATABASE_URL en el archivo .env (contraseña oculta).

9.3 Primera migración

Con el modelo y la conexión listos se ejecutó la migración inicial:

```

npm install @prisma/client@6
npx prisma migrate dev --name "init"

```

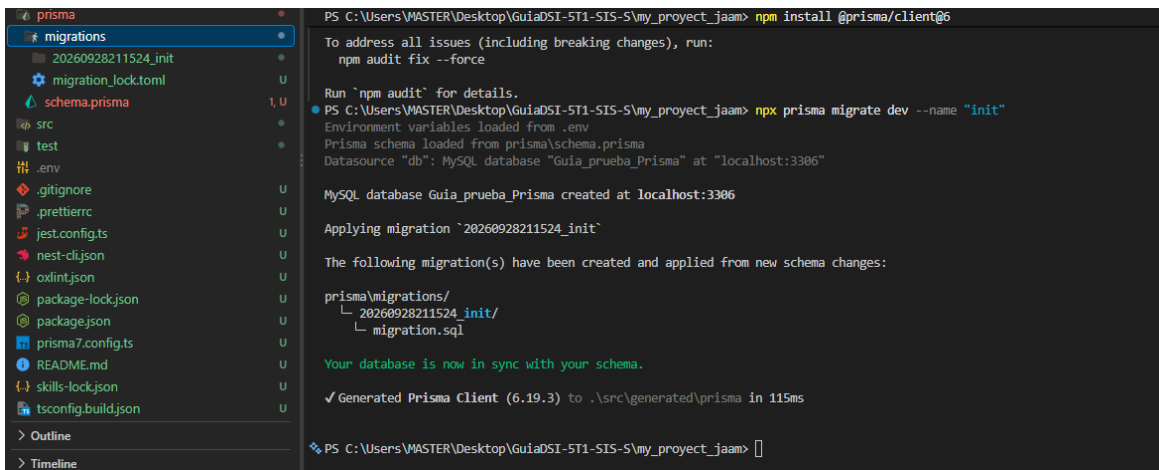


Figura 13. Instalación de @prisma/client y ejecución de la migración init.

Prisma leyó las variables de entorno y el esquema, creó la base de datos `Guia_prueba_Prisma` en `localhost:3306`, aplicó la migración `20260928211524_init` y generó el cliente de Prisma (versión 6.19.3) en `src/generated/prisma`.

```

Run `npm audit` for details.
● PS C:\Users\MASTER\Desktop\GuiaDSI-5T1-SIS-S\my_proyect_jaam> npx prisma migrate dev --name "init"
Environment variables loaded from .env
Prisma schema loaded from prisma\schema.prisma
Datasource "db": MySQL database "Guia_prueba_Prisma" at "localhost:3306"

MySQL database Guia_prueba_Prisma created at localhost:3306

Applying migration `20260928211524_init`

The following migration(s) have been created and applied from new schema changes:

prisma\migrations/
├─ 20260928211524_init/
│   └─ migration.sql

Your database is now in sync with your schema.

✓ Generated Prisma Client (6.19.3) to .\src\generated\prisma in 115ms

```

Figura 14. Detalle de la salida: base de datos creada, migración aplicada y cliente generado.

Se creó la carpeta `prisma/migrations/`, que contiene una subcarpeta por cada migración con su archivo `migration.sql` (los cambios realizados) y el archivo `migration_lock.toml`.

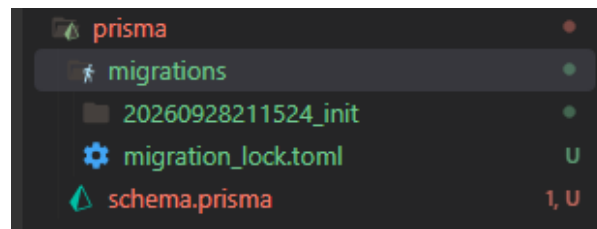


Figura 15. Carpeta prisma/migrations con la migración 20260928211524_init.

Observación: el proyecto se inicializó con archivos de Prisma 7 (`prisma7.config.ts`), pero la sintaxis `url = env(...)` dentro del datasource corresponde a Prisma 6; por eso se instaló `@prisma/client@6` y el cliente se generó en versión 6.19.3.

10. Relaciones entre tablas

Una vez migrada la primera tabla se crea una segunda que se relaciona con `User`. Los tipos de relación a nivel de base de datos son:

10.1 Relación uno a uno (1 a 1)

- Un usuario tiene un solo perfil.
- Un perfil pertenece a un solo usuario.
- Se usa `@unique` en la clave foránea.

```

17 model User {
18   id          Int          @id @default(autoincrement())
19   profile     Profile?
20 }
21
22 model Profile {
23   id          Int          @id @default(autoincrement())
24   userId      Int          @unique
25   user        User         @relation(fields: [userId], references: [id])
26 }
27
28

```

Figura 16. Modelos User y Profile con relación uno a uno.

En el modelo Profile, el campo userId es la clave foránea con @unique, y la relación se declara con @relation(fields: [userId], references: [id]). En User, el campo profile Profile? es opcional.

10.2 Relación uno a muchos (1 a N)

- Un tenant tiene muchos usuarios.
- Un usuario pertenece a un solo tenant.
- El lado “1” usa un arreglo (User[]).
- El lado “N” usa un campo escalar (tenantId).

10.3 Relación muchos a muchos (N a N)

Se presenta en la guía como referencia de los tipos de relación disponibles; no se implementa en este proyecto.

10.4 Modelo Tenant

Para el proyecto se requiere una nueva tabla Tenant, de modo que cada usuario tenga asignado un tenant específico y un tenant pueda tener varios usuarios (relación 1 a N). Tras ajustar el modelo se ejecuta la migración:

```
npx prisma migrate dev --name "add_tenant_model"
```

```
PS C:\Users\MASTER\Desktop\GuiadSI-5T1-SIS-S\my_proyect_jaam> npx prisma migrate dev --name "add_tenant_model"

[-] Removed foreign key on columns (ownerId)

[*] Changed the `dailyschedule` table
[-] Removed foreign key on columns (scheduleId)

[*] Changed the `defaultcalendar` table
[-] Removed foreign key on columns (calendarId)
[-] Removed foreign key on columns (userId)

[*] Changed the `meeting` table
[-] Removed foreign key on columns (defaultConnectedCalendarId)
[-] Removed foreign key on columns (ownerId)
[-] Removed foreign key on columns (scheduleId)

[*] Changed the `schedule` table
[-] Removed foreign key on columns (ownerId)

[*] Changed the `session` table
[-] Removed foreign key on columns (userId)

We need to reset the MySQL database "Guia_prueba_Prisma" at "localhost:3306"

You may use prisma migrate reset to drop the development database.
All data will be lost.
PS C:\Users\MASTER\Desktop\GuiadSI-5T1-SIS-S\my_proyect_jaam>
```

Figura 17. Prisma solicita un reset de la base de datos al ejecutar `add_tenant_model`.

En esta ejecución Prisma detectó diferencias entre el historial de migraciones y el estado real de la base de datos (aparecen cambios sobre tablas como `meeting`, `schedule`, `session` y `dailyschedule`) y solicitó un reset de `Guia_prueba_Prisma`.

10.5 Verificación en MySQL Workbench

La base de datos se puede explorar con cualquier cliente (la guía recomienda TablePlus; aquí se usó MySQL Workbench). La consulta a `_prisma_migrations` muestra la migración `20260928211524_init` aplicada correctamente, junto con las tablas existentes en el esquema `guia_prueba_prisma`.

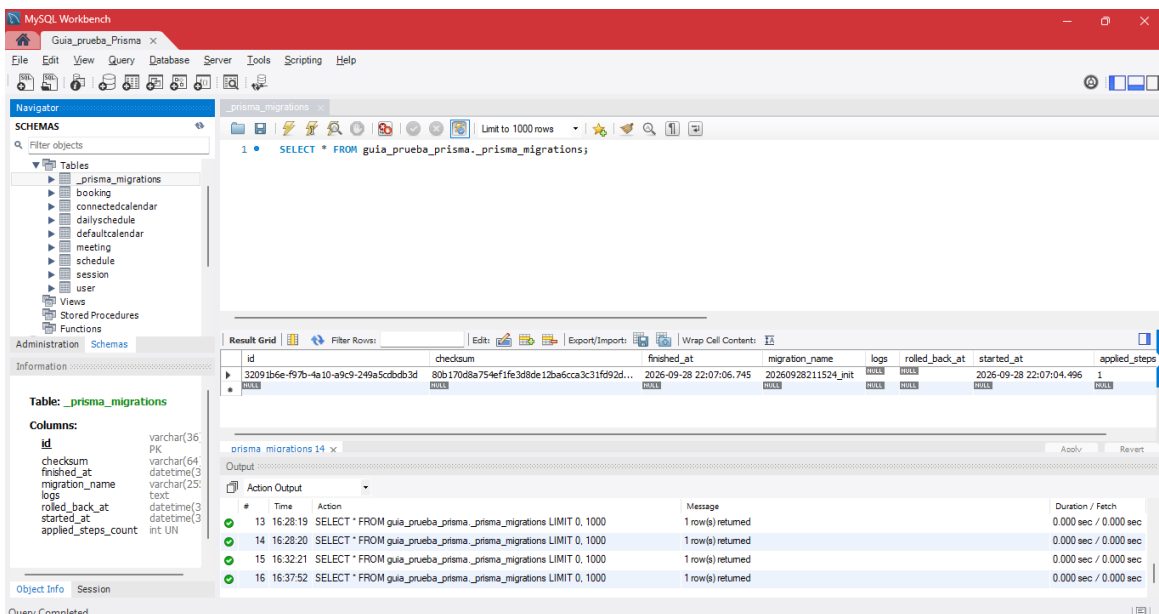


Figura 18. MySQL Workbench: tabla `_prisma_migrations` y listado de tablas del esquema.

Punto de control: en este punto la guía solicita realizar el primer commit que evidencie el avance

11. Seeder (datos de prueba)

Un seed ("sembrado") es un conjunto de datos de prueba que se insertan en la base de datos para desarrollo y testing, demostraciones de la aplicación y configuración inicial del sistema.

11.1 Pasos realizados

- Crear el archivo `seed.ts` dentro de la carpeta `prisma`.
- Registrar el comando del seed en `package.json`.
- Ajustar el `schema.prisma`.
- Instalar las dependencias necesarias y ejecutar el seed desde la terminal.

11.2 Archivo `seed.ts` y configuración en `package.json`

Se creó el archivo `prisma/seed.ts`, que contiene la lógica de inserción de los datos de prueba. Para que Prisma sepa cómo ejecutarlo, se agregó en `package.json` la propiedad `prisma.seed` con el comando `ts-node prisma/seed.ts`:

```
"prisma": {
  "seed": "ts-node prisma/seed.ts"
}
```

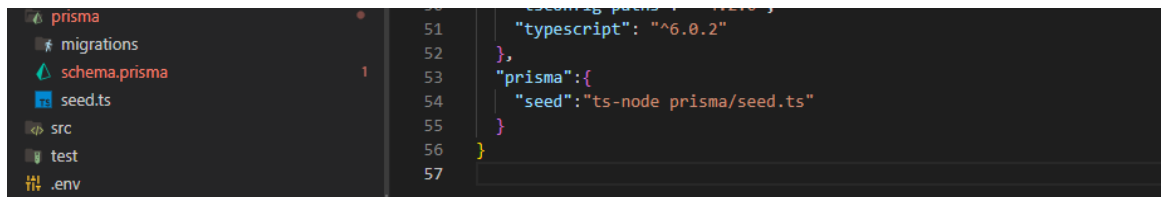


Figura 19. Archivo `prisma/seed.ts` y configuración del comando seed en `package.json`.

11.3 Ajuste del `schema.prisma`

En el bloque `generator` del esquema se cambió el proveedor a `prisma-client-js`, manteniendo la ruta de salida `../src/generated/prisma`:

```
generator client {
  provider = "prisma-client-js"
  output   = "../src/generated/prisma"
}
```

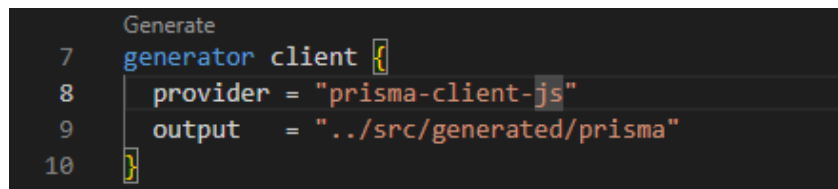
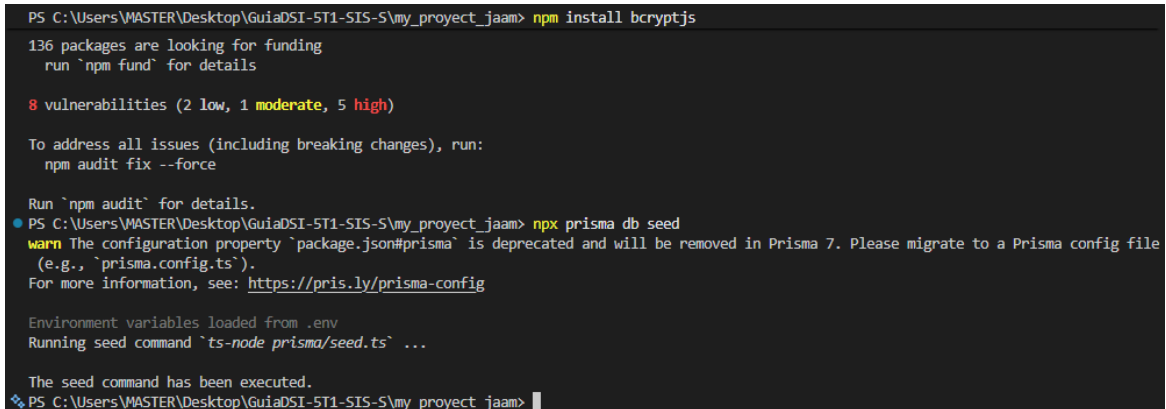


Figura 20. Bloque `generator` con el proveedor `prisma-client-js`.

11.4 Instalación de `bcryptjs` y ejecución del seed

Se instaló `bcryptjs`, librería utilizada para cifrar (hash) las contraseñas de los usuarios de prueba, y luego se ejecutó el seed:

```
npm install bcryptjs
npx prisma db seed
```



```
PS C:\Users\MASTER\Desktop\GuiadSI-ST1-SIS-S\my_proyect_jaam> npm install bcryptjs
136 packages are looking for funding
  run `npm fund` for details

8 vulnerabilities (2 low, 1 moderate, 5 high)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\MASTER\Desktop\GuiadSI-ST1-SIS-S\my_proyect_jaam> npx prisma db seed
warn The configuration property `package.json#prisma` is deprecated and will be removed in Prisma 7. Please migrate to a Prisma config file
(e.g., `prisma.config.ts`).
For more information, see: https://pris.ly/prisma-config

Environment variables loaded from .env
Running seed command `ts-node prisma/seed.ts` ...

The seed command has been executed.
PS C:\Users\MASTER\Desktop\GuiadSI-ST1-SIS-S\my_proyect_jaam>
```

Figura 21. Instalación de bcryptjs y ejecución de npx prisma db seed.

La terminal cargó las variables de entorno desde `.env`, ejecutó el comando `ts-node prisma/seed.ts` y finalizó con el mensaje “**The seed command has been executed.**”, lo que confirma que el proceso de inserción se completó.

11.5 Validación de los datos

Tras la inserción, los datos se validan con el cliente de base de datos utilizado (MySQL Workbench) o con Prisma Studio, que permite consultar el esquema y los datos desde el navegador:

```
npx prisma studio
```

Guía 2

12. Módulo y servicio de Prisma

Extraer el `PrismaClient` a un servicio propio se considera una buena práctica, ya que permite que el cliente de Prisma se instancie una sola vez en la aplicación y se reutilice desde cualquier módulo.

12.1 Generación de los archivos

```
nest generate module prisma
nest generate service prisma
```

Los comandos crean la carpeta `src/prisma` con los archivos `prisma.module.ts` y `prisma.service.ts`. El archivo `prisma.service.spec.ts` puede eliminarse, ya que no se usará en esta guía.

12.2 PrismaService

El servicio importa `PrismaClient` y lo extiende, de modo que `PrismaService` hereda todos los métodos de acceso a la base de datos:

```
@Injectable()
export class PrismaService extends PrismaClient {}
```

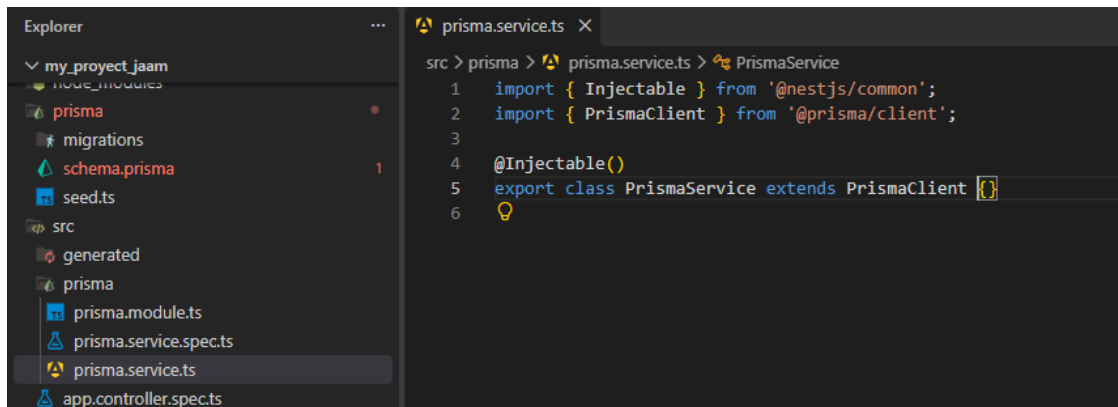


Figura 1. PrismaService: clase que extiende PrismaClient.

12.3 PrismaModule

En el módulo se registra PrismaService en providers y se agrega a exports. Con esto se comparte una única instancia (singleton) del servicio con los módulos que importen PrismaModule:

```

@Module({
  providers: [PrismaService],
  exports: [PrismaService]
})
export class PrismaModule {}

```

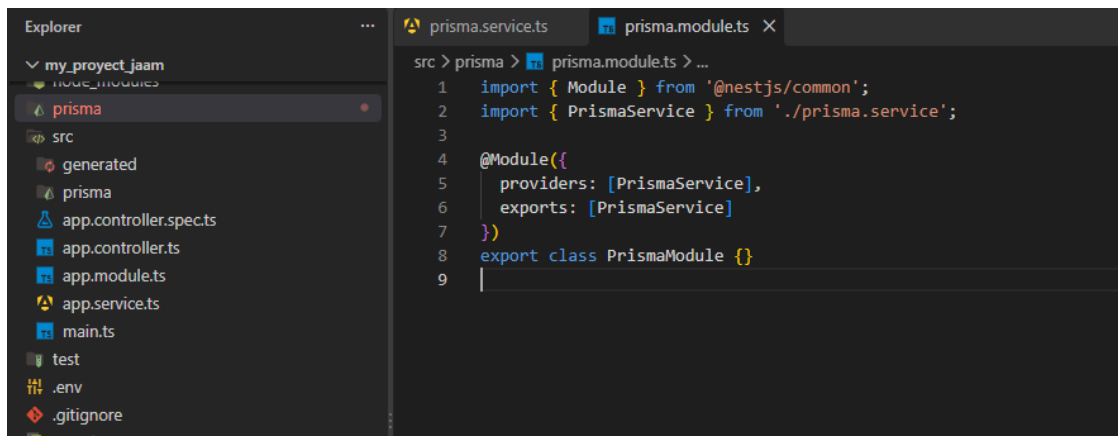


Figura 2. PrismaModule con PrismaService en providers y exports.

13. Documentación de la API con Swagger

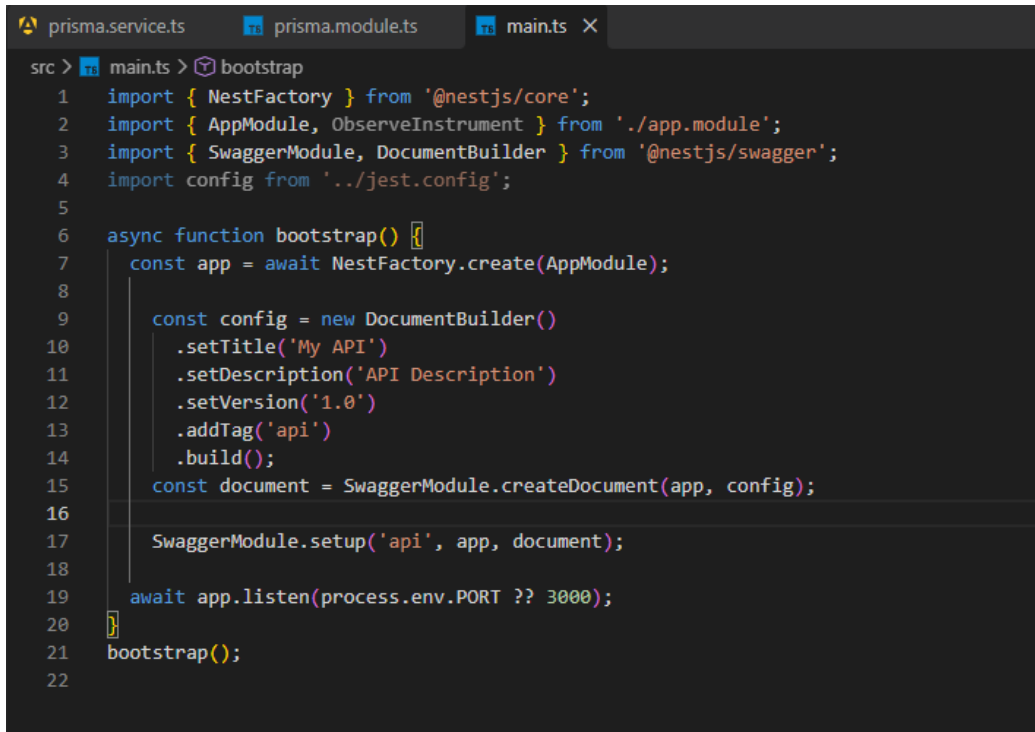
Swagger (especificación OpenAPI, <https://swagger.io/specification/>) permite documentar la aplicación y probar los endpoints desde el navegador. Se instalaron las dependencias:

```

npm install --save @nestjs/swagger swagger-ui-express

```

Luego se ajustó el archivo `main.ts` para construir el documento con `DocumentBuilder` (título, descripción, versión y etiqueta) y montar la interfaz en la ruta `api`:

The image shows a code editor with three tabs: 'prisma.service.ts', 'prisma.module.ts', and 'main.ts'. The 'main.ts' tab is active, showing a TypeScript file. At the top, there's a command prompt 'src > main.ts > bootstrap'. The code starts with imports for NestFactory, AppModule, SwaggerModule, and DocumentBuilder. It then defines an async bootstrap function that creates the app, builds a Swagger configuration, creates the document, sets up the Swagger module, and listens on a port. The code is as follows:

```
src > main.ts > bootstrap
1  import { NestFactory } from '@nestjs/core';
2  import { AppModule, ObserveInstrument } from './app.module';
3  import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
4  import config from '../jest.config';
5
6  async function bootstrap() {
7    const app = await NestFactory.create(AppModule);
8
9    const config = new DocumentBuilder()
10     .setTitle('My API')
11     .setDescription('API Description')
12     .setVersion('1.0')
13     .addTag('api')
14     .build();
15    const document = SwaggerModule.createDocument(app, config);
16
17    SwaggerModule.setup('api', app, document);
18
19    await app.listen(process.env.PORT ?? 3000);
20  }
21  bootstrap();
22
```

Figura 3. main.ts con la configuración de Swagger.

- `DocumentBuilder` define los metadatos de la documentación: título, descripción, versión y etiqueta.
- `SwaggerModule.createDocument` genera el documento a partir de la aplicación.
- `SwaggerModule.setup('api', app, document)` publica la interfaz en `/api`.
- La aplicación escucha en el puerto definido por `PORT` o, por defecto, en el 3000.

Al levantar la aplicación y abrir `http://localhost:3000/api` se muestra la interfaz de Swagger, desde donde se puede repetir la petición de la guía anterior (GET `/`) y probar el resto de rutas.

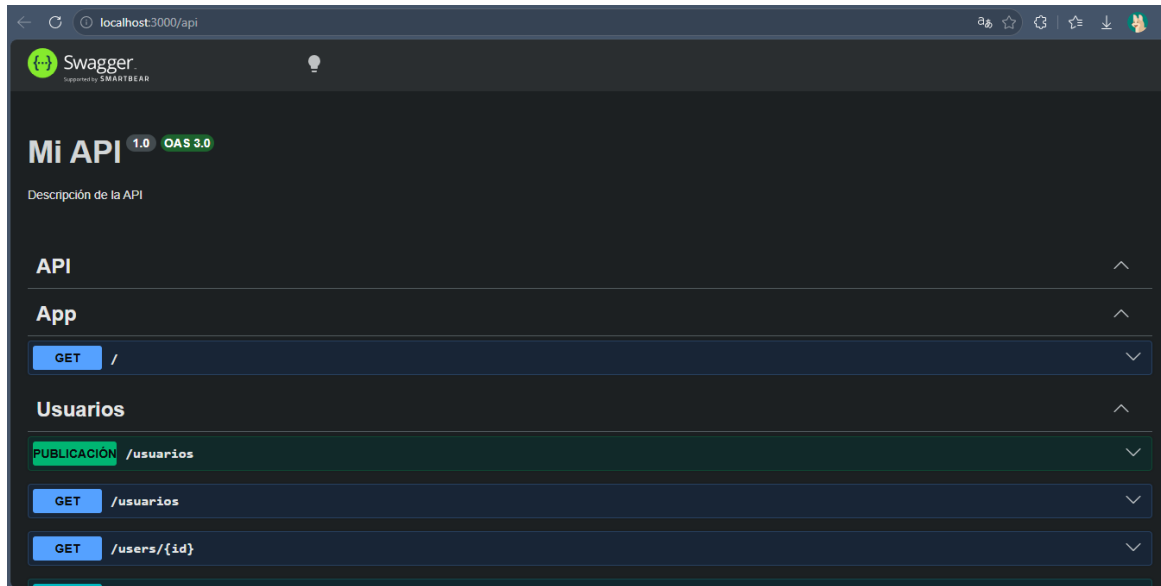


Figura 4. Interfaz de Swagger en localhost:3000/api.

Observaciones: (1) En `main.ts` se generó automáticamente la línea `import config from './jest.config'`, que no se utiliza y puede eliminarse. (2) El título y la descripción de la captura de Swagger ("Mi API") difieren de los del código ("My API"), y algunas rutas visibles (`/usuarios`, `/users/{id}`) no coinciden con las del recurso `users` generado más adelante.

14. Recurso CRUD de usuarios

Nest CLI puede generar un recurso completo con un solo comando:

```
nest generate resource
```

El asistente solicita el nombre del recurso (en plural), el tipo de capa de transporte y si se desea generar la plantilla completa del CRUD. Se respondió:

- **Nombre:** `users`
- **Capa de transporte:** REST API
- **Generar puntos de entrada CRUD:** Yes

```
Problems 8 Output Debug Console Terminal Ports
● PS C:\Users\MASTER\Desktop\GuiadSI-5T1-SIS-S\my_proyect_jaam> Nest generate resource
✓ What name would you like to use for this resource (plural, e.g., "users")? users
✓ What transport layer do you use? REST API
✓ Would you like to generate CRUD entry points? Yes
CREATE src/users/users.controller.ts (928 bytes)
CREATE src/users/users.controller.spec.ts (586 bytes)
CREATE src/users/users.module.ts (257 bytes)
CREATE src/users/users.service.ts (635 bytes)
CREATE src/users/users.service.spec.ts (471 bytes)
CREATE src/users/dto/create-user.dto.ts (31 bytes)
CREATE src/users/dto/update-user.dto.ts (168 bytes)
CREATE src/users/entities/user.entity.ts (22 bytes)
UPDATE src/app.module.ts (856 bytes)
○ PS C:\Users\MASTER\Desktop\GuiadSI-5T1-SIS-S\my_proyect_jaam>
```

Figura 5. Ejecución de nest generate resource y archivos creados.

Nest creó el controlador, el servicio, el módulo, los archivos de prueba, los DTO (create-user.dto.ts y update-user.dto.ts) y la entidad (user.entity.ts), y actualizó app.module.ts para registrar el nuevo módulo.

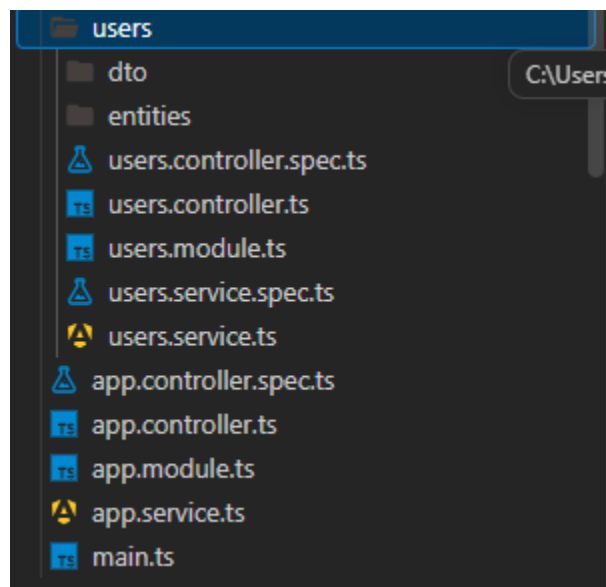


Figura 6. Carpeta src/users con los archivos generados.

15. Inyección de Prisma en el módulo de usuarios

Para usar PrismaService dentro del recurso de usuarios primero se importa PrismaModule en UsersModule:

```
@Module({
  controllers: [UsersController],
  providers: [UsersService],
  imports: [PrismaModule]
})
```

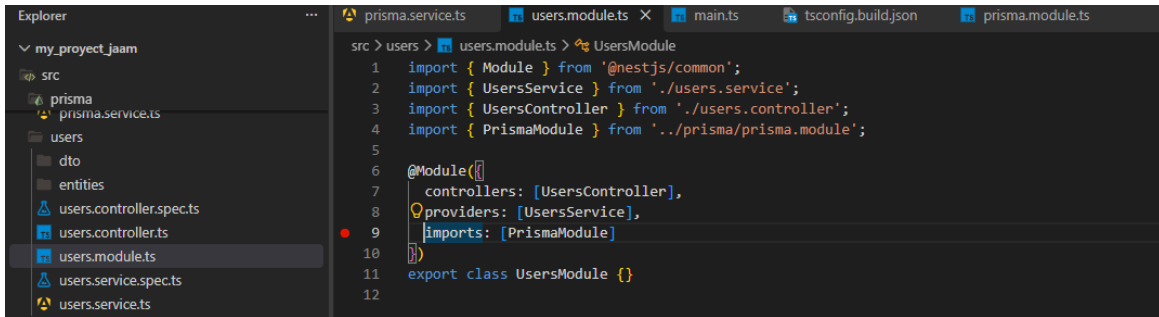


Figura 7. UsersModule con PrismaModule en imports.

Después se inyecta PrismaService en el constructor de UsersService, lo que permite acceder a la base de datos desde el servicio:

```
constructor(private readonly prisma: PrismaService) {}
```

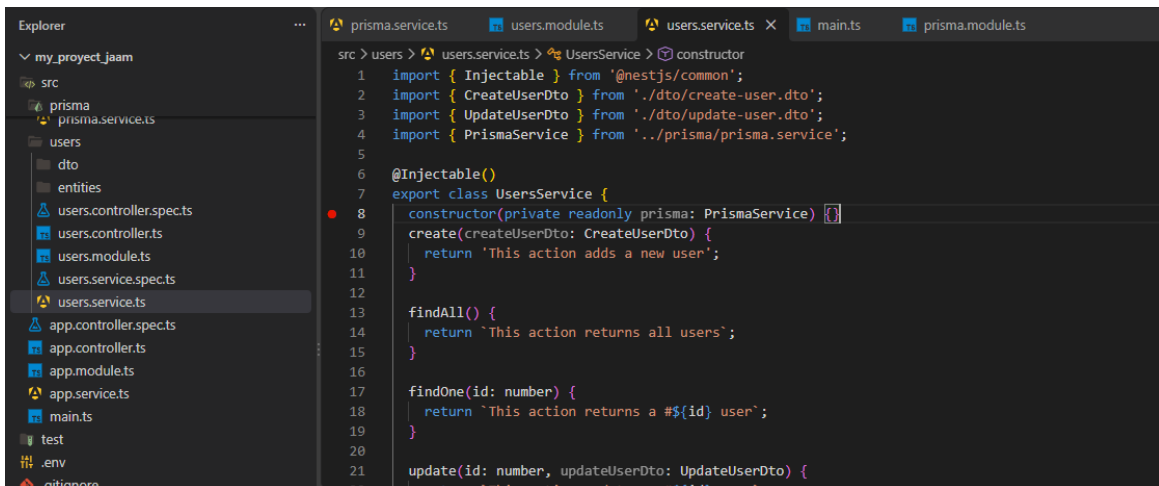


Figura 8. UsersService con PrismaService inyectado en el constructor.

Concepto clave – inyección de dependencias: en lugar de crear manualmente el PrismaService con new, Nest lo construye y lo entrega automáticamente al declararlo en el constructor. Así, el servicio depende de una abstracción compartida y se puede sustituir fácilmente (por ejemplo, en pruebas).

16. Operaciones del CRUD

Cada método del servicio, que inicialmente devolvía un texto de plantilla, se reemplazó por una consulta real con Prisma. Después de guardar cada cambio, se prueba desde Swagger.

16.1 Listar usuarios (GET /users)

findAll devuelve todos los registros de la tabla de usuarios con findMany():

```

findAll() {
  return this.prisma.user.findMany();
}

```

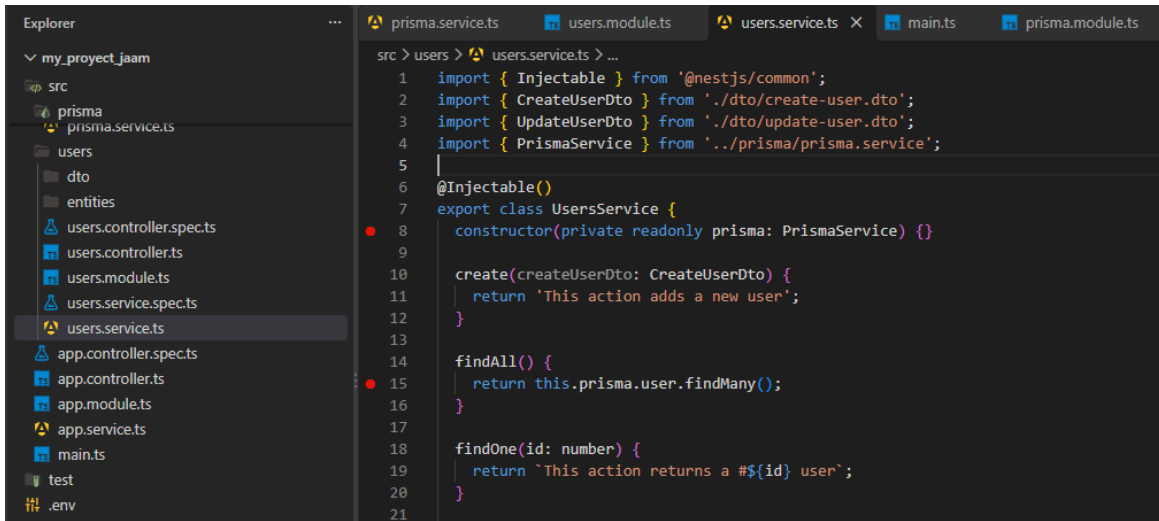


Figura 9. Método `findAll` implementado con `prisma.user.findMany()`.

16.2 Buscar un usuario (GET /users/:id)

`findOne` recibe el identificador y debe devolver un único registro. En la captura de esta guía el método aún conserva el texto de plantilla; la implementación esperada con Prisma es:

```

findOne(id: number) {
  return this.prisma.user.findUnique({ where: { id } });
}

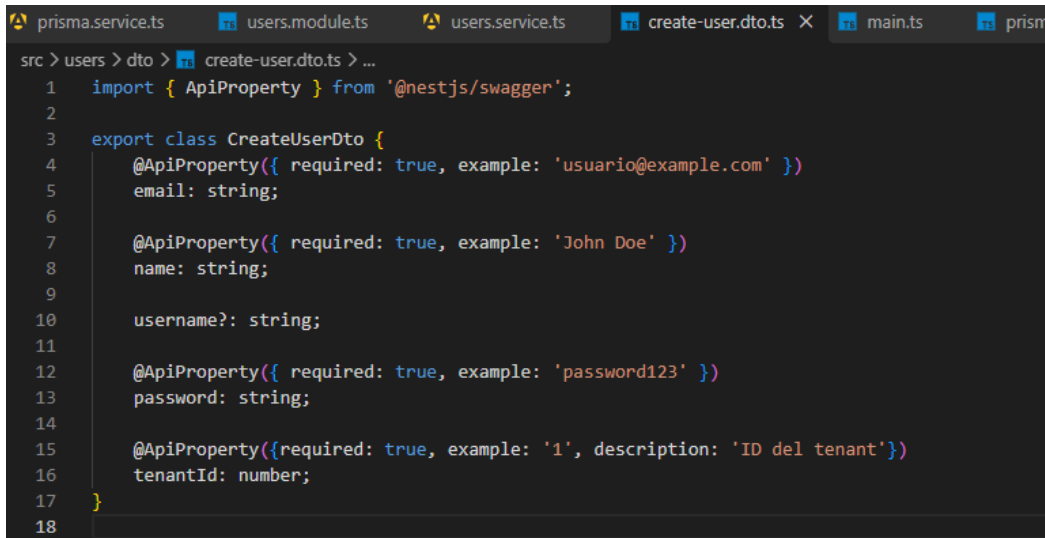
```

16.3 Crear un usuario (POST /users)

En NestJS, un **DTO (Data Transfer Object)** es una clase que define la estructura de los datos que se envían entre el cliente y el servidor. Actúa como un “contrato” que garantiza la correcta transmisión de los datos y se usa comúnmente junto con `class-validator` para definir reglas de validación, evitando errores por datos con formato inesperado.

El `CreateUserDto` define los campos requeridos para crear un usuario y utiliza el decorador `@ApiProperty` de Swagger para documentarlos, con un ejemplo de valor:

- `email` (string, requerido)
- `name` (string, requerido)
- `username` (string, opcional)
- `password` (string, requerido)
- `tenantId` (number, requerido): identificador del tenant al que pertenece el usuario, según la relación 1 a N definida en la Guía N° 1.



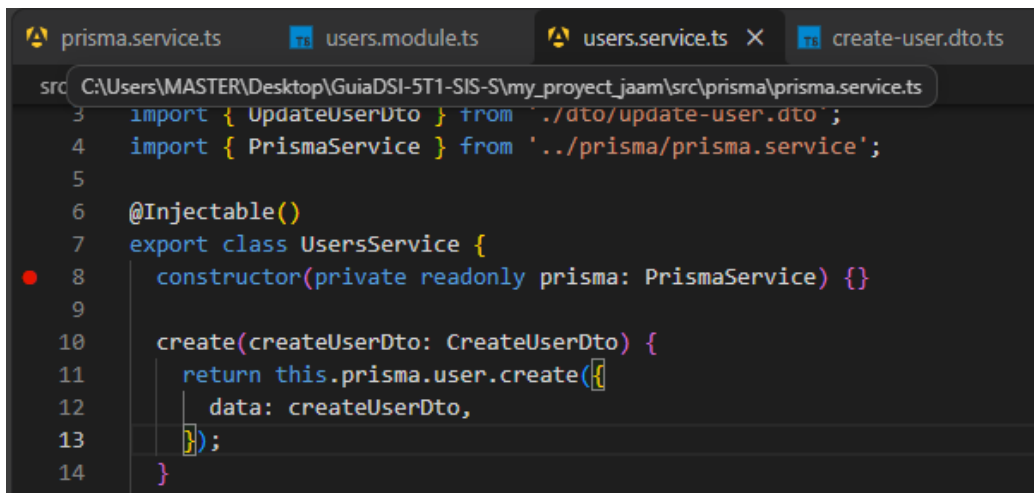
```
prisma.service.ts  users.module.ts  users.service.ts  create-user.dto.ts  main.ts  prism

src > users > dto > create-user.dto.ts > ...
1  import { ApiProperty } from '@nestjs/swagger';
2
3  export class CreateUserDto {
4    @ApiProperty({ required: true, example: 'usuario@example.com' })
5    email: string;
6
7    @ApiProperty({ required: true, example: 'John Doe' })
8    name: string;
9
10   username?: string;
11
12   @ApiProperty({ required: true, example: 'password123' })
13   password: string;
14
15   @ApiProperty({ required: true, example: '1', description: 'ID del tenant' })
16   tenantId: number;
17 }
18
```

Figura 10. CreateUserDto con los campos del usuario y decoradores @ApiProperty.

El método create recibe el DTO y lo pasa directamente a Prisma:

```
create(createUserDto: CreateUserDto) {
  return this.prisma.user.create({
    data: createUserDto,
  });
}
```



```
prisma.service.ts  users.module.ts  users.service.ts  create-user.dto.ts

src C:\Users\MASTER\Desktop\GuiaDSI-5T1-SIS-S\my_proyect_jaam\src\prisma\prisma.service.ts
3  import { UpdateUserDto } from '../dto/update-user.dto';
4  import { PrismaService } from '../prisma/prisma.service';
5
6  @Injectable()
7  export class UsersService {
8    constructor(private readonly prisma: PrismaService) {}
9
10   create(createUserDto: CreateUserDto) {
11     return this.prisma.user.create({
12       data: createUserDto,
13     });
14   }
15 }
```

Figura 11. Método create implementado con prisma.user.create().

Recomendación: el DTO recibe la contraseña en texto plano y la guarda tal cual. En un sistema real debe cifrarse antes de guardarla (por ejemplo, con bcryptjs, como en el seed de la Guía N° 1) y validarse los datos con class-validator.

16.4 Actualizar un usuario (PATCH /users/:id)

update localiza el registro por su id y actualiza los campos recibidos en el DTO:

```
update(id: number, updateUserDto: UpdateUserDto) {
  return this.prisma.user.update({
    where: { id },
```

```

    data: updateUserDto,
  });
}

```

```

26   update(id: number, updateUserDto: UpdateUserDto) {
27     return this.prisma.user.update({
28       where: { id },
29       data: updateUserDto,
30     });
31   }
32

```

Figura 12. Método update implementado con prisma.user.update().

16.5 Eliminar un usuario (DELETE /users/:id)

remove elimina el registro cuyo id coincide con el recibido:

```

remove(id: number) {
  return this.prisma.user.delete({
    where: { id },
  });
}

```

```

33   remove(id: number) {
34     return this.prisma.user.delete({
35       where: { id },
36     });
37   }
38

```

Figura 13. Método remove implementado con prisma.user.delete().

17. Pruebas en Swagger

Cada endpoint del CRUD puede ejecutarse desde <http://localhost:3000/api> con el botón “Try it out”: listar usuarios, buscar por id, crear con un cuerpo JSON basado en el DTO, actualizar y eliminar. Los resultados también pueden verificarse en la base de datos con MySQL Workbench o Prisma Studio.

Guía 3

18. Introducción y objetivo

La Guía Didáctica N° 3 cierra el proyecto base de las guías anteriores con la creación del servicio de autenticación. El módulo que se construye se encarga de atender las peticiones de inicio de sesión (login): recibe un correo electrónico y una contraseña, verifica en la base de datos que el usuario exista y, si las credenciales son correctas, devuelve un token JWT firmado que el cliente enviará en las solicitudes posteriores para acceder a las rutas protegidas.

Concepto clave – JWT: JSON Web Token es un estándar abierto (RFC 7519) que define un token compacto y firmado para transmitir información entre el cliente y el servidor. En esta guía el token contiene el id, el email y el role del usuario, junto con una firma generada con una clave secreta; así el servidor puede comprobar que el token no fue alterado sin consultar la base de datos en cada petición.

19. Creación del módulo de autenticación

19.1 Generación de los archivos

Desde la raíz del proyecto se ejecutaron los siguientes comandos de Nest CLI:

```
nest generate module auth
nest generate service auth
nest generate controller auth
```

Cada comando crea sus archivos y actualiza `auth.module.ts` para registrar el servicio y el controlador (Figura 1).

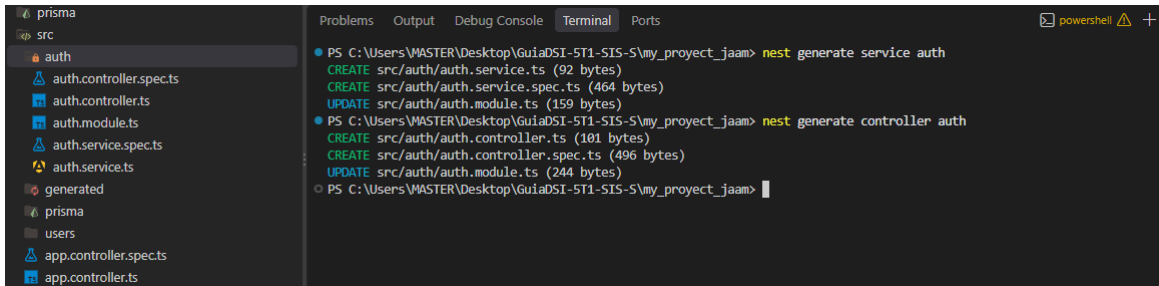


Figura 1. Salida de `nest generate service auth` y `nest generate controller auth`.

El resultado es la carpeta `src/auth` con los archivos `auth.module.ts`, `auth.service.ts` y `auth.controller.ts`. Los archivos `spec.ts` generados se eliminaron, ya que no se usarán en esta guía (Figura 2). Además, Nest importó de forma automática el nuevo módulo en el `AppModule` principal.

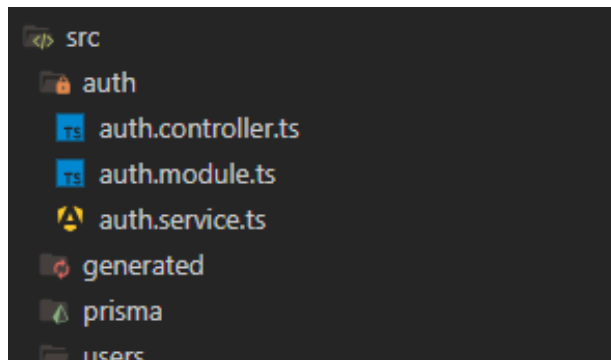
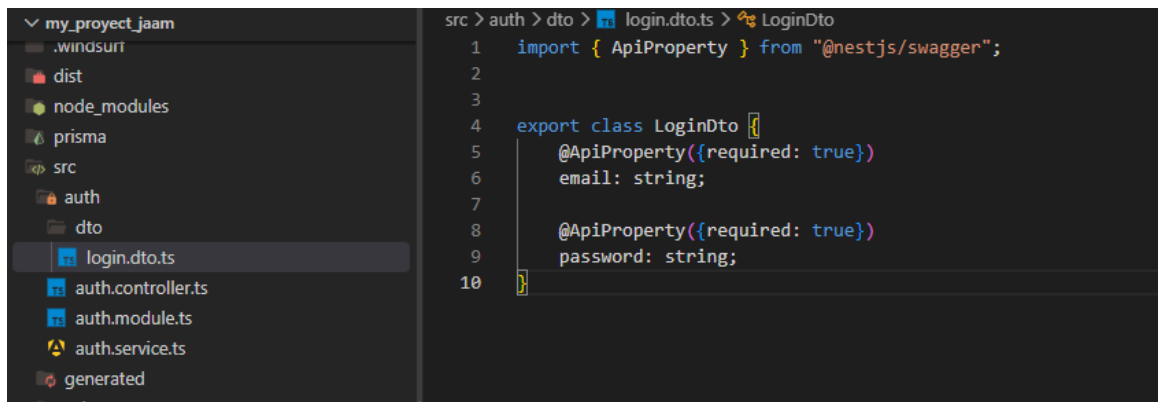


Figura 2. Estructura de la carpeta `src/auth` tras eliminar los archivos `spec.ts`.

20. DTO y controlador de login

20.1 LoginDto

Antes de configurar la petición de login se creó un DTO que ayuda a definir los datos que debe enviar el cliente. Dentro de la carpeta `auth` se creó la carpeta `dto` con el archivo `login.dto.ts`, que contiene los campos requeridos `email` y `password`, documentados con el decorador `@ApiProperty` de Swagger.

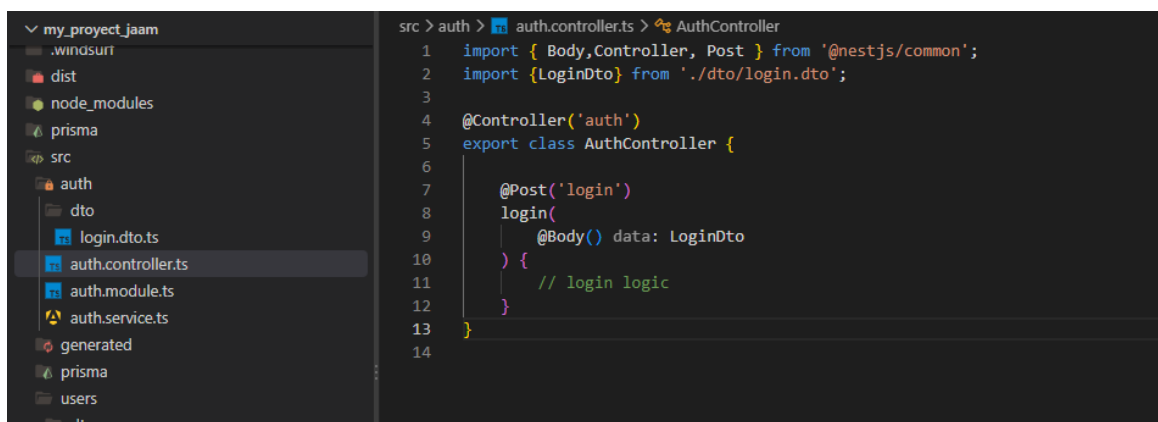


```
src > auth > dto > login.dto.ts > LoginDto
1  import { ApiProperty } from "@nestjs/swagger";
2
3
4  export class LoginDto {
5      @ApiProperty({required: true})
6      email: string;
7
8      @ApiProperty({required: true})
9      password: string;
10 }
```

Figura 3. Archivo auth/dto/login.dto.ts con los campos email y password.

20.2 Endpoint de login

En `AuthController` se configuró la petición `POST /auth/login`, que recibe el `LoginDto` en el cuerpo de la solicitud mediante `@Body()`. En este punto el método solo contiene un comentario donde irá la lógica de autenticación.



```
src > auth > auth.controller.ts > AuthController
1  import { Body, Controller, Post } from '@nestjs/common';
2  import { LoginDto } from '../dto/login.dto';
3
4  @Controller('auth')
5  export class AuthController {
6
7      @Post('login')
8      login(
9          @Body() data: LoginDto
10      ) {
11          // login logic
12      }
13  }
14
```

Figura 4. AuthController con la petición de login y el LoginDto como cuerpo.

Al levantar la aplicación, el nuevo endpoint puede verificarse desde Swagger en `http://localhost:3000/api`, donde se muestra el cuerpo de ejemplo con los campos `email` y `password`.



Figura 5. Endpoint POST /auth/login visible en la interfaz de Swagger.

Con esto la estructura queda lista. El siguiente paso es crear el servicio y definir el método de autenticación: se recibe un correo y una contraseña y se verifica en la base de datos si el usuario existe.

21. Instalación de dependencias

Antes de implementar el servicio se instalaron, desde la raíz del proyecto, las dependencias que facilitan el manejo de JWT y de las estrategias de autenticación:

```
npm i @nestjs/jwt
npm i @nestjs/passport
npm i passport passport-jwt
```

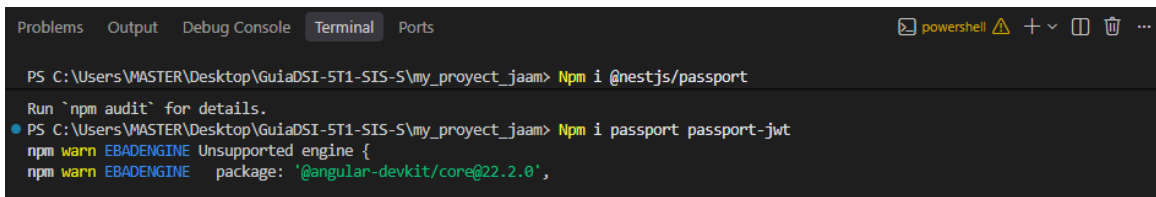


Figura 6. Instalación de las dependencias de autenticación desde la terminal.

- **@nestjs/jwt**: módulo oficial de NestJS para el manejo de tokens JWT. Permite firmar (sign) y verificar (verify) tokens y se integra fácilmente con los módulos de autenticación de Nest.
- **@nestjs/passport**: adaptador oficial que permite usar Passport.js dentro de NestJS. Facilita la creación de estrategias de autenticación (JWT, Local, Google, etc.) y se encarga de integrar Passport con los guards (AuthGuard) de Nest.
- **passport**: núcleo de Passport.js; gestiona las estrategias y las sesiones.
- **passport-jwt**: estrategia específica que indica a Passport cómo manejar los tokens JWT: extraerlos, validarlos y decodificarlos.

22. Configuración de AuthModule

En `auth.module.ts` se importó `JwtModule` y se registró con `JwtModule.register()`, definiendo el secreto con el que se firman los tokens (`secret`) y su tiempo de expiración (`signOptions: { expiresIn: '1h' }`), es decir, una hora.

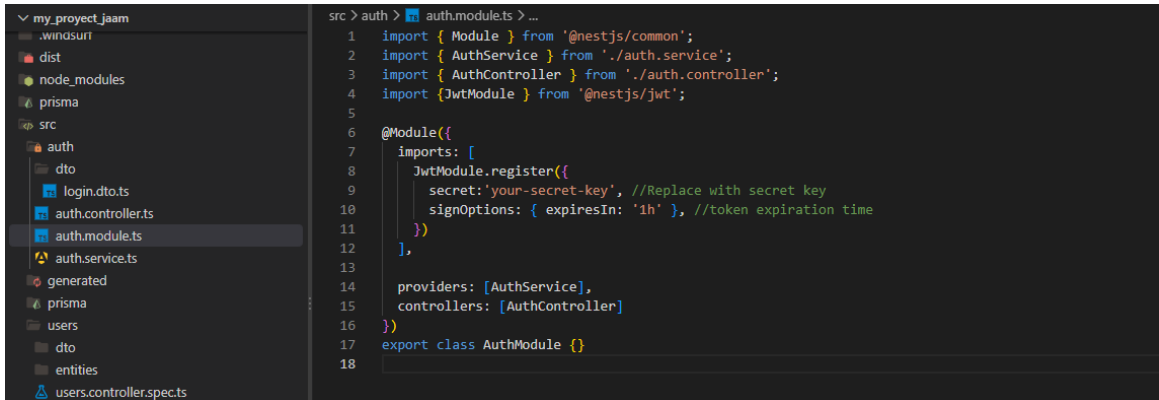


Figura 7. AuthModule con JwtModule configurado (secreto y expiración).

Recomendación: el valor `your-secret-key` es solo un ejemplo y debe reemplazarse por una clave propia. El secreto es la pieza que garantiza la validez de los tokens, por lo que en un proyecto real no debe escribirse directamente en el código, sino leerse de una variable de entorno del archivo `.env` (que no debe publicarse en el repositorio). Además, debe ser exactamente el mismo en el módulo y en la estrategia JWT (sección 25); si difieren, los tokens firmados serán rechazados.

23. Servicio de autenticación

23.1 Inyección de dependencias

En el constructor de `AuthService` se inyectaron `JwtService`, proporcionado por `@nestjs/jwt`, y `PrismaService`. Este último se utiliza para validar que el correo que envía el cliente exista en la base de datos. También se importó `bcryptjs`, la misma librería usada en el seed, para comparar contraseñas cifradas.

23.2 Método validateUser

Se creó la función asíncrona `validateUser`, que recibe el `LoginDto` y sigue estos pasos:

- Busca en la base de datos, con `findUnique`, al usuario cuyo correo coincide con el recibido y lo guarda en `foundUser`.
- Evalúa el contenido de `foundUser`: si no existe un correo registrado, devuelve `null`.
- Compara con `bcrypt.compare` la contraseña recibida con la almacenada del usuario encontrado.
- Si la contraseña coincide, firma el JWT con el método `sign` de `JwtService`; el token contiene el `id`, el `email` y el `role` del usuario. Si no coincide, lanza una `UnauthorizedException` con el mensaje "Credenciales inválidas".

```

src > auth > auth.service.ts > AuthService > validateUser > foundUser > where > email
1  import { Injectable, UnauthorizedException } from '@nestjs/common';
2  import { JwtService } from '@nestjs/jwt';
3  import { LoginDto } from '../dto/login.dto';
4  import { PrismaService } from '../../prisma/prisma.service';
5  import * as bcrypt from 'bcryptjs';
6
7  @Injectable()
8  export class AuthService {
9    constructor(
10     private jwtService: JwtService,
11     private prisma: PrismaService) {}
12
13   async validateUser(user: LoginDto) {
14     const foundUser = await this.prisma.user.findUnique({
15       where: {
16         email: user.email
17       }
18     });
19
20     if (!foundUser) return null;
21
22     const isValidPassword = await bcrypt.compare(user.password, foundUser.password);
23
24     if (isValidPassword) {
25       return this.jwtService.sign({
26         id: foundUser.id,
27         email: foundUser.email,
28         role: foundUser.role,
29       });
30     } else {
31       throw new UnauthorizedException('Credenciales inválidas');
32     }
33   }
34 }

```

Figura 8. AuthService: constructor con JwtService y PrismaService inyectados, y método validateUser.

23.3 Uso desde el controlador

Se regresó a `AuthController` para inyectar `AuthService` en el constructor y usar la función recién creada. El método `login` llama a `validateUser` con los datos recibidos; si el resultado es `null`, lanza una `HttpException` con el estado 401 (UNAUTHORIZED), y en caso contrario devuelve el token.

```

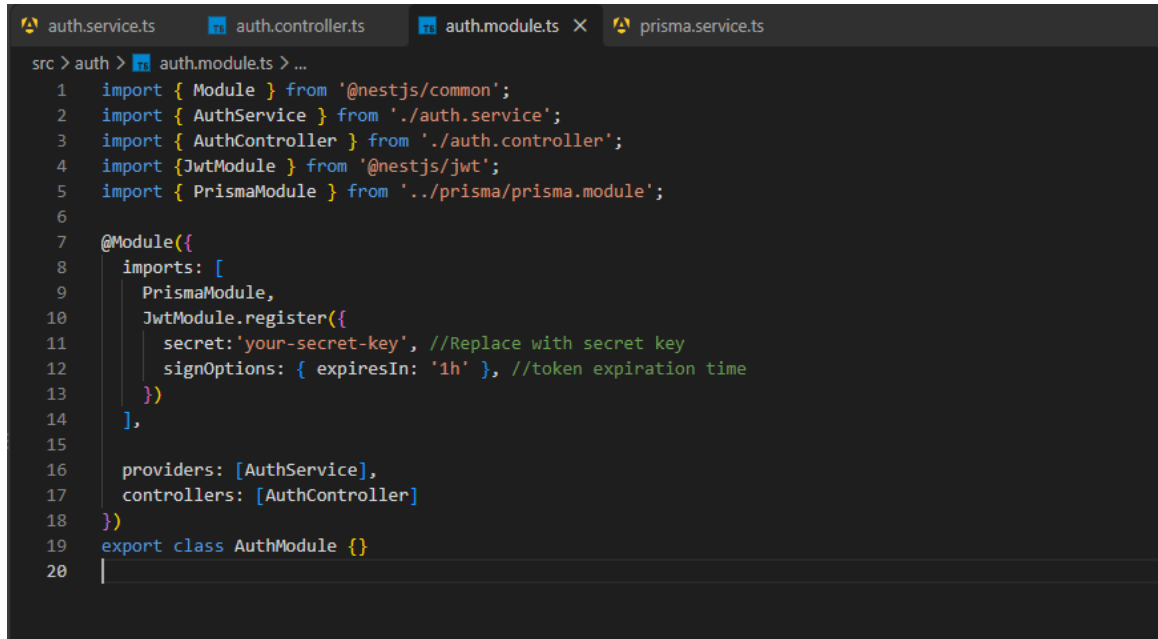
src > auth > auth.controller.ts > ...
1  import { Body, Controller, Post, HttpException, HttpStatus } from '@nestjs/common';
2  import { LoginDto } from '../dto/login.dto';
3  import { AuthService } from '../auth.service';
4
5  @Controller('auth')
6  export class AuthController {
7
8    constructor(private authService: AuthService) {}
9
10   @Post('login')
11   async login(
12     @Body() data: LoginDto
13   ) {
14     const usertoken = await this.authService.validateUser(data);
15
16     if (!usertoken) throw new HttpException('Credenciales inválidas', HttpStatus.UNAUTHORIZED);
17
18     return usertoken;
19   }
20 }
21

```

Figura 9. AuthController con AuthService inyectado y uso de validateUser.

23.4 Error por dependencia faltante

Al ejecutar la aplicación apareció un error: Nest no pudo resolver la dependencia de `PrismaService` dentro de `AuthModule`, porque `PrismaModule` no había sido importado en ese módulo. Es el mismo principio aplicado en la sección 15 con `UsersModule`: un módulo solo puede inyectar los providers que exporta otro módulo si lo importa. La solución fue agregar `PrismaModule` en la lista de `imports` de `AuthModule`.



```
src > auth > auth.module.ts > ...
1  import { Module } from '@nestjs/common';
2  import { AuthService } from './auth.service';
3  import { AuthController } from './auth.controller';
4  import { JwtModule } from '@nestjs/jwt';
5  import { PrismaModule } from '../prisma/prisma.module';
6
7  @Module({
8    imports: [
9      PrismaModule,
10     JwtModule.register({
11       secret: 'your-secret-key', //Replace with secret key
12       signOptions: { expiresIn: '1h' }, //token expiration time
13     })
14   ],
15
16   providers: [AuthService],
17   controllers: [AuthController]
18 })
19 export class AuthModule {}
20
```

Figura 10. AuthModule con PrismaModule agregado en imports.

24. Prueba del servicio y verificación del token

Con la aplicación funcionando se envió una solicitud de login desde Swagger con el correo y la contraseña de un usuario registrado. El servidor respondió con código 201 y el JWT firmado en el cuerpo de la respuesta.

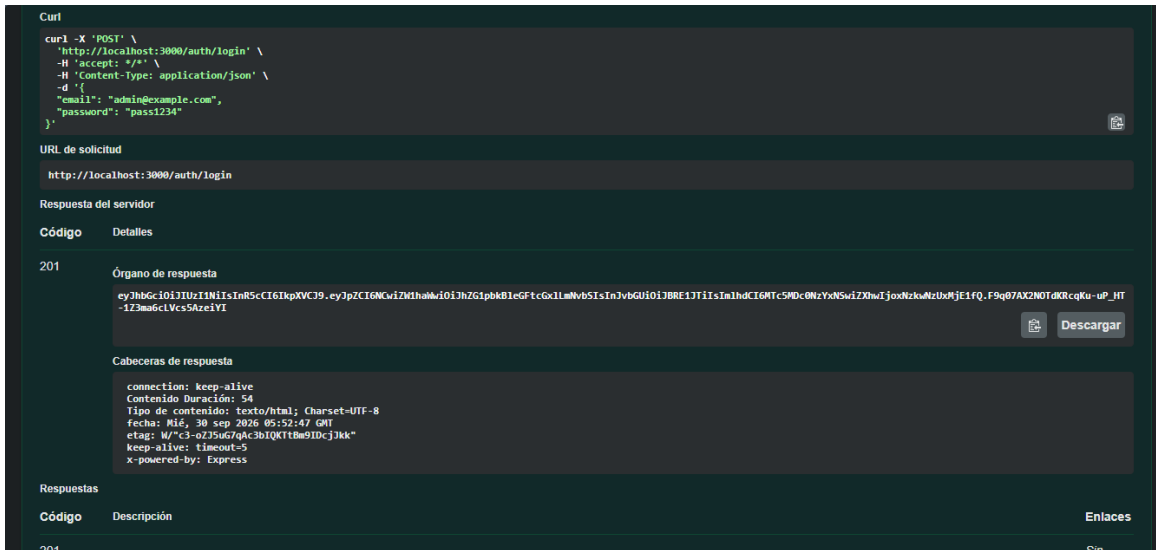


Figura 11. Petición de login en Swagger y token JWT devuelto en la respuesta.

El token puede pegarse en la herramienta de decodificación de <https://www.jwt.io/>, que muestra el contenido del token (id, email, role y las fechas de emisión y expiración) y permite comprobar la firma.

Observación: un JWT está firmado, pero no cifrado: cualquiera que lo tenga puede leer su contenido en herramientas como jwt.io. Por eso el payload no debe incluir datos sensibles, como la contraseña.

25. Estrategia JWT

Una vez firmado y validado el token, falta configurar que este sea enviado en las solicitudes futuras y que el servidor lo verifique. Para ello se crea una estrategia de autenticación con passport-jwt.

25.1 Soporte de TypeScript

Primero se instaló el soporte de tipos de TypeScript para passport-jwt:

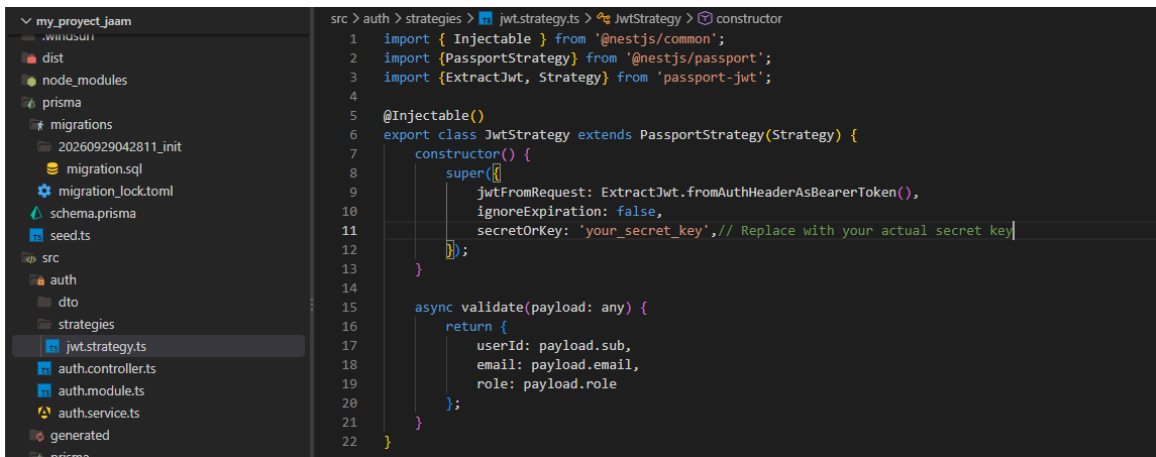
```
npm i --save-dev @types/passport-jwt
```

25.2 Creación de la estrategia

Con el soporte instalado se creó el archivo `jwt.strategy.ts` dentro de la carpeta `auth/strategies`. La clase `JwtStrategy` extiende `PassportStrategy(Strategy)` y define las siguientes propiedades en el constructor:

- **jwtFromRequest:** indica de dónde se extrae el token; con `ExtractJwt.fromAuthHeaderAsBearerToken()` se toma del encabezado `Authorization` con el formato Bearer.
- **ignoreExpiration:** se establece en `false` para que la estrategia no ignore la expiración del token y rechace los tokens vencidos.
- **secretOrKey:** clave con la que se verifica la firma del token. Debe ser la misma definida en `JwtModule`; en un proyecto real debe definirse utilizando variables de entorno.

Además, el método `validate` recibe el payload ya verificado y devuelve los datos del usuario (email y role, junto con su identificador), que Nest deja disponibles en la solicitud.

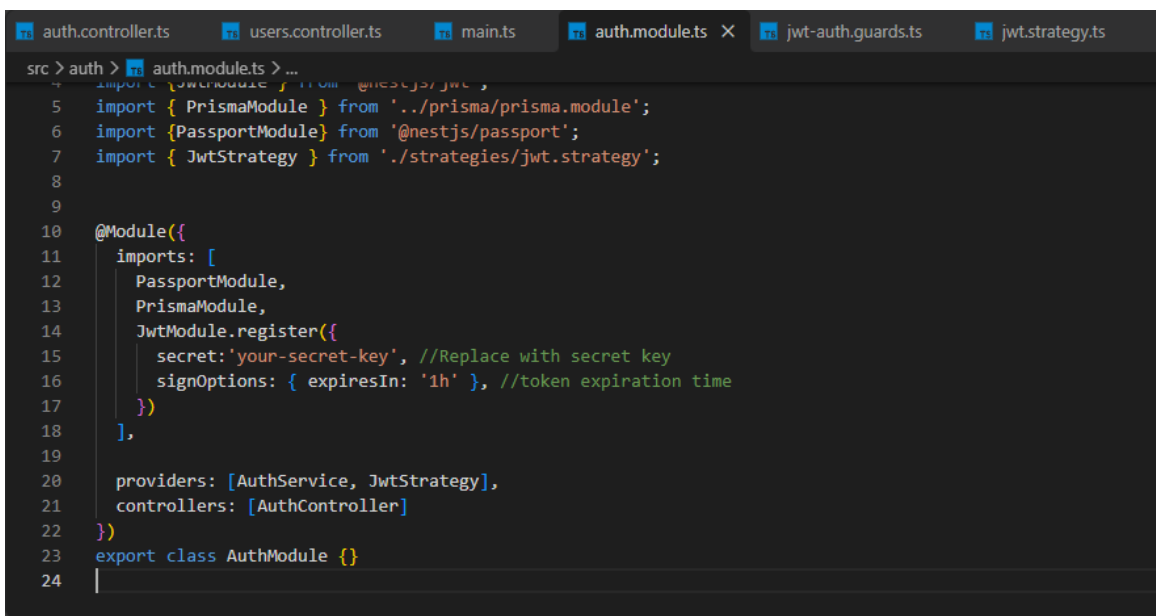


```
src > auth > strategies > jwt.strategy.ts > JwtStrategy > constructor
1 import { Injectable } from '@nestjs/common';
2 import { PassportStrategy } from '@nestjs/passport';
3 import { ExtractJwt, Strategy } from 'passport-jwt';
4
5 @Injectable()
6 export class JwtStrategy extends PassportStrategy(Strategy) {
7   constructor() {
8     super({
9       jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
10      ignoreExpiration: false,
11      secretOrKey: 'your_secret_key', // Replace with your actual secret key
12    });
13   }
14
15   async validate(payload: any) {
16     return {
17       userId: payload.sub,
18       email: payload.email,
19       role: payload.role
20     };
21   }
22 }
```

Figura 12. Estrategia JWT con jwtFromRequest, ignoreExpiration, secretOrKey y el método validate.

25.3 Registro de la estrategia en AuthModule

En AuthModule se agregó JwtStrategy en providers y PassportModule en imports, junto a PrismaModule y JwtModule.

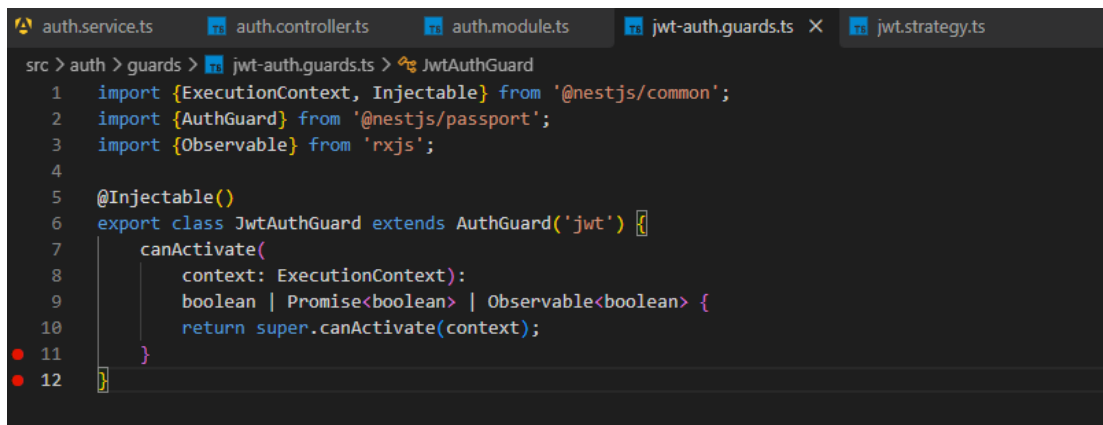


```
src > auth > auth.module.ts > ...
1 import { Module } from '@nestjs/common';
2 import { JwtModule } from '@nestjs/jwt';
3 import { PassportModule } from '@nestjs/passport';
4 import { PrismaModule } from '../prisma/prisma.module';
5 import { JwtStrategy } from '../strategies/jwt.strategy';
6
7 @Module({
8   imports: [
9     PassportModule,
10    PrismaModule,
11    JwtModule.register({
12      secret: 'your-secret-key', //Replace with secret key
13      signOptions: { expiresIn: '1h' }, //token expiration time
14    }),
15  ],
16   providers: [AuthService, JwtStrategy],
17   controllers: [AuthController]
18 })
19 export class AuthModule {}
```

Figura 13. AuthModule con PassportModule en imports y JwtStrategy en providers.

26. Protección de rutas con Guards

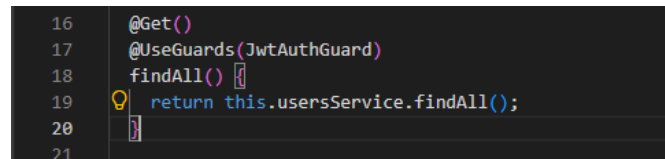
Para proteger las rutas se utilizan los Guards de Nest (<https://docs.nestjs.com/guards>), clases que determinan si una solicitud puede llegar al controlador según una condición; en este caso, que incluya un token válido. Se creó la carpeta guards dentro de auth con el archivo jwt-auth.guards.ts, que define la clase JwtAuthGuard. Esta extiende AuthGuard('jwt'), por lo que usa la estrategia creada en la sección anterior, y su método canActivate delega la validación en la clase base.



```
src > auth > guards > jwt-auth.guards.ts > JwtAuthGuard
1 import {ExecutionContext, Injectable} from '@nestjs/common';
2 import {AuthGuard} from '@nestjs/passport';
3 import {Observable} from 'rxjs';
4
5 @Injectable()
6 export class JwtAuthGuard extends AuthGuard('jwt') {
7   canActivate(
8     context: ExecutionContext):
9     boolean | Promise<boolean> | Observable<boolean> {
10     return super.canActivate(context);
11   }
12 }
```

Figura 14. Archivo `auth/guards/jwt-auth.guards.ts` con la clase `JwtAuthGuard`.

Para comprobar que funciona, se decoró el método `findAll` del controlador de usuarios con `@UseGuards(JwtAuthGuard)` y se agregaron las importaciones correspondientes. Con esto la ruta exige un token válido para responder.



```
16 @Get()
17 @UseGuards(JwtAuthGuard)
18 findAll() {
19   return this.userService.findAll();
20 }
21
```

Figura 15. Método `findAll` de `UsersController` protegido con `JwtAuthGuard`.

27. Integración del token en Swagger

Para no enviar el token manualmente en cada prueba, se agregó a Swagger el soporte de autenticación con token desde `main.ts`. En el `DocumentBuilder` se añadió `addBearerAuth` con el tipo `http`, el esquema `bearer`, el formato `JWT` y el encabezado `Authorization`, y se registró el requisito de seguridad con `addSecurityRequirements`.

```

src > main.ts > ...
1  import { NestFactory } from '@nestjs/core';
2  import { AppModule, ObserveInstrument } from './app.module';
3  import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
4
5  async function bootstrap() {
6      const app = await NestFactory.create(AppModule);
7
8      const config = new DocumentBuilder()
9          .setTitle('My API')
10         .setDescription('API Description')
11         .setVersion('1.0')
12         .addTag('api')
13         .addBearerAuth({
14             type: 'http',
15             scheme: 'bearer',
16             bearerFormat: 'JWT',
17             in: 'header',
18             name: 'Authorization',
19             description: 'Enter your bearer token',
20         })
21         .addSecurityRequirements('bearerAuth')
22         .build();
23
24         const document = SwaggerModule.createDocument(app, config);
25
26         SwaggerModule.setup('api', app, document);
27
28         await app.listen(process.env.PORT ?? 3000);
29     }
30     bootstrap();
31

```

Figura 16. main.ts con la configuración de autenticación Bearer en Swagger.

Como resultado, la interfaz de Swagger muestra el botón “Autorizar” en la parte superior y un candado en las rutas protegidas.

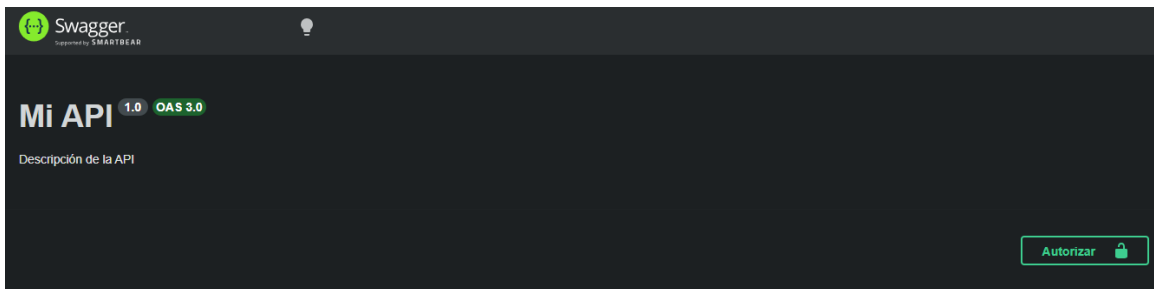


Figura 17. Interfaz de Swagger con el botón Autorizar.

27.1 Generación del token

Se generó un token mediante la petición de login (Figura 11) y se copió el valor devuelto en la respuesta.

27.2 Uso del token en Swagger

El token copiado se ingresó en el campo “Valor” de la ventana de autorizaciones disponibles y se confirmó con “Autorizar”. A partir de ese momento Swagger incluye el JWT en el encabezado de cada petición.

Curl

```
curl -X 'GET' \
  'http://localhost:3000/users' \
  -H 'accept: */*'
```

URL de solicitud

http://localhost:3000/users

Respuesta del servidor

Código	Detalles
401	Error. No autorizado

401 No documentado

Órgano de respuesta

```
{
  "message": "Unauthorized",
  "statusCode": 401
}
```

Descargar

Cabeceras de respuesta

```
connection: keep-alive
Duración del contenido: 43
content-type: application/json; charset=utf-8
fecha: Mié, 30 sep 2026 07:22:19 GMT
etag: W/"2b-dGnJzt6gv1nJjX6D39RztDMptng"
keep-alive: timeout=5
x-powered-by: Express
```

Respuestas

Código	Descripción	Enlaces
200		Sin enlaces

Figura 20. Petición GET /users sin token y respuesta 401.